# Machine-Level Programming V: Advanced Topics

CSE 238/2038/2138: Systems Programming

## Instructor:

Birol Gençyılmaz & Zuhal Altuntaş

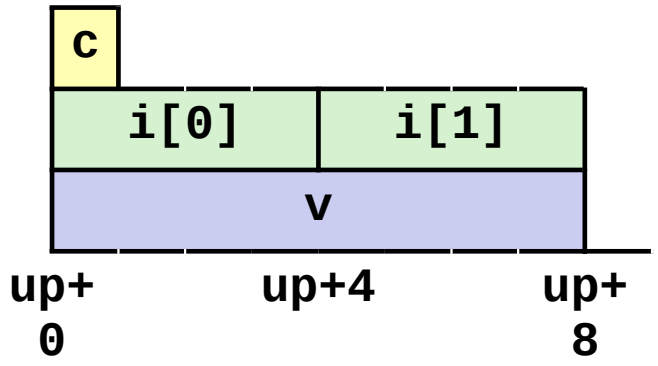
*Slides adapted from Bryant & O'Hallaron's slides*

# Union Allocation

- Allocate according to largest element
- Can only use one field at a time

```
union U1 {
  char c;
  int i[2];
  double v;
} *up;
```

```
struct S1 {
  char c;
  int i[2];
  double v;
} *sp;
```






# Example 1

```
typedef union {
    struct {
        long   u;
        short  v;
        char   w;
    } t1;
    struct {
        int a[2];
        char  *p;
    } t2;
} u_type;
```

You write a series of functions of the form

```
void get(u_type *up, type *dest) {
    *dest =  expr;
}
```

with different access expressions *expr* and with destination data type *type* set according to type associated with *expr*. You then examine the code generated when compiling the functions to see if they match your expectations.

Suppose in these functions that `up` and `dest` are loaded into registers `%rdi` and `%rsi`, respectively. Fill in the following table with data type *type* and sequences of one to three instructions to compute the expression and store the result at `dest`.

Suppose you are given the job of checking that a C compiler generates the proper code for structure and union access. You write the following structure declaration:

| EXPR | TYPE | Code |
|---|---|---|
| `up->t1.u` | `long` | `movq (%rdi), %rax`<br>`movq %rax, (%rsi)` |
| `up->t1.v` | | |
| `&up->t1.w` | | |
| `up->t2.a` | | |
| `up->t2.a[up->t1.u]` | | |
| `*up->t2.p` | | |

# Example 1

```
typedef union {
    struct {
        long   u;
        short  v;
        char   w;
    } t1;
    struct {
        int a[2];
        char  *p;
    } t2;
} u_type;
```

Suppose you are given the job of checking that a C compiler generates the proper code for structure and union access. You write the following structure declaration:

| EXPR | TYPE | Code |
|---|---|---|
| `up->t1.u` | `long` | `movq (%rdi), %rax`<br>`movq %rax, (%rsi)` |
| `up->t1.v` | `short` | `movw 8(%rdi), %ax`<br>`movw %ax, (%rsi)` |
| `&up->t1.w` | | |
| `up->t2.a` | | |
| `up->t2.a[up->t1.u]` | | |
| `*up->t2.p` | | |

You write a series of functions of the form

```
void get(u_type *up, type *dest) {
    *dest =  expr;
}
```

with different access expressions *expr* and with destination data type *type* set according to type associated with *expr*. You then examine the code generated when compiling the functions to see if they match your expectations.

Suppose in these functions that `up` and `dest` are loaded into registers `%rdi` and `%rsi`, respectively. Fill in the following table with data type *type* and sequences of one to three instructions to compute the expression and store the result at `dest`.

# Example 1

```
typedef union {
    struct {
        long   u;
        short  v;
        char   w;
    } t1;
    struct {
        int a[2];
        char  *p;
    } t2;
} u_type;
```

You write a series of functions of the form

```
void get(u_type *up, type *dest) {
    *dest =  expr;
}
```

with different access expressions *expr* and with destination data type *type* set according to type associated with *expr*. You then examine the code generated when compiling the functions to see if they match your expectations.

Suppose in these functions that `up` and `dest` are loaded into registers `%rdi` and `%rsi`, respectively. Fill in the following table with data type *type* and sequences of one to three instructions to compute the expression and store the result at `dest`.

Suppose you are given the job of checking that a C compiler generates the proper code for structure and union access. You write the following structure declaration:

| EXPR | TYPE | Code |
|---|---|---|
| `up->t1.u` | `long` | `movq (%rdi), %rax`<br>`movq %rax, (%rsi)` |
| `up->t1.v` | `short` | `movw 8(%rdi), %ax`<br>`movw %ax, (%rsi)` |
| `&up->t1.w` | `char *` | `addq $10, %rdi`<br>`movq %rdi, (%rsi)` |
| `up->t2.a` | | |
| `up->t2.a[up->t1.u]` | | |
| `*up->t2.p` | | |

# Example 1

```
typedef union {
    struct {
        long   u;
        short  v;
        char   w;
    } t1;
    struct {
        int a[2];
        char  *p;
    } t2;
} u_type;
```

Suppose you are given the job of checking that a C compiler generates the proper code for structure and union access. You write the following structure declaration:

| EXPR | TYPE | Code |
|---|---|---|
| `up->t1.u` | `long` | `movq (%rdi), %rax`<br>`movq %rax, (%rsi)` |
| `up->t1.v` | `short` | `movw 8(%rdi), %ax`<br>`movw %ax, (%rsi)` |
| `&up->t1.w` | `char *` | `addq $10, %rdi`<br>`movq %rdi, (%rsi)` |
| `up->t2.a` | `int *` | `movq %rdi, (%rsi)` |
| `up->t2.a[up->t1.u]` | | |
| `*up->t2.p` | | |

You write a series of functions of the form

```
void get(u_type *up, type *dest) {
    *dest =  expr;
}
```

with different access expressions *expr* and with destination data type *type* set according to type associated with *expr*. You then examine the code generated when compiling the functions to see if they match your expectations.

Suppose in these functions that `up` and `dest` are loaded into registers `%rdi` and `%rsi`, respectively. Fill in the following table with data type *type* and sequences of one to three instructions to compute the expression and store the result at `dest`.

# Example 1

```
typedef union {
    struct {
        long   u;
        short  v;
        char   w;
    } t1;
    struct {
        int a[2];
        char  *p;
    } t2;
} u_type;
```

You write a series of functions of the form

```
void get(u_type *up, type *dest) {
    *dest =  expr;
}
```

with different access expressions *expr* and with destination data type *type* set according to type associated with *expr*. You then examine the code generated when compiling the functions to see if they match your expectations.

Suppose in these functions that `up` and `dest` are loaded into registers `%rdi` and `%rsi`, respectively. Fill in the following table with data type *type* and sequences of one to three instructions to compute the expression and store the result at `dest`.

Suppose you are given the job of checking that a C compiler generates the proper code for structure and union access. You write the following structure declaration:

| EXPR | TYPE | Code |
|---|---|---|
| `up->t1.u` | `long` | `movq (%rdi), %rax`<br>`movq %rax, (%rsi)` |
| `up->t1.v` | `short` | `movw 8(%rdi), %ax`<br>`movw %ax, (%rsi)` |
| `&up->t1.w` | `char *` | `addq $10, %rdi`<br>`movq %rdi, (%rsi)` |
| `up->t2.a` | `int *` | `movq %rdi, (%rsi)` |
| `up->t2.a[up->t1.u]` | `int` | `movq (%rdi), %rax`<br>`movl (%rdi,%rax,4), %eax`<br>`movl %eax, (%rsi)` |
| `*up->t2.p` | | |

# Example 1

```
typedef union {
    struct {
        long   u;
        short  v;
        char   w;
    } t1;
    struct {
        int a[2];
        char  *p;
    } t2;
} u_type;
```

You write a series of functions of the form

```
void get(u_type *up, type *dest) {
    *dest =  expr;
}
```

with different access expressions *expr* and with destination data type *type* set according to type associated with *expr*. You then examine the code generated when compiling the functions to see if they match your expectations.

Suppose in these functions that `up` and `dest` are loaded into registers `%rdi` and `%rsi`, respectively. Fill in the following table with data type *type* and sequences of one to three instructions to compute the expression and store the result at `dest`.

Suppose you are given the job of checking that a C compiler generates the proper code for structure and union access. You write the following structure declaration:

| EXPR | TYPE | Code |
|---|---|---|
| `up->t1.u` | `long` | `movq (%rdi), %rax`<br>`movq %rax, (%rsi)` |
| `up->t1.v` | `short` | `movw 8(%rdi), %ax`<br>`movw %ax, (%rsi)` |
| `&up->t1.w` | `char *` | `addq $10, %rdi`<br>`movq %rdi, (%rsi)` |
| `up->t2.a` | `int *` | `movq %rdi, (%rsi)` |
| `up->t2.a[up->t1.u]` | `int` | `movq (%rdi), %rax`<br>`movl (%rdi,%rax,4), %eax`<br>`movl %eax, (%rsi)` |
| `*up->t2.p` | `char` | `movq 8(%rdi), %rax`<br>`movb (%rax), %al`<br>`movb %al, (%rsi)` |

# Example 2

Consider the following union declaration:

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next;
9       } e2;
10  };
```

A. What are the offsets (in bytes) of the following fields:

```
e1.p      _____
e1.y      _____
e2.x      _____
e2.next   _____
```

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```
1   void proc (union ele *up) {
2       up->_____ = *(_____) - _____;
3   }
```

# Example 2

Consider the following union declaration:

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next;
9       } e2;
10  };
```

A. What are the offsets (in bytes) of the following fields:

| | |
|---|---|
| e1.p | 0 |
| e1.y | |
| e2.x | |
| e2.next | |

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```
1   void proc (union ele *up) {
2       up->_________ = *(_________) - _________;
3   }
```

# Example 2

Consider the following union declaration:

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next;
9       } e2;
10  };
```

A. What are the offsets (in bytes) of the following fields:

| | |
|---|---|
| e1.p | 0 |
| e1.y | 8 |
| e2.x | |
| e2.next | |

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```
1   void proc (union ele *up) {
2       up->_________ = *(_________) - _________;
3   }
```

# Example 2

Consider the following union declaration:

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next;
9       } e2;
10  };
```

A. What are the offsets (in bytes) of the following fields:

| | |
|---|---|
| e1.p | 0 |
| e1.y | 8 |
| e2.x | 0 |
| e2.next | |

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```
1   void proc (union ele *up) {
2       up->_________ = *(_________) - _________;
3   }
```

# Example 2

Consider the following union declaration:

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next;
9       } e2;
10  };
```

A. What are the offsets (in bytes) of the following fields:

| | |
|---|---|
| e1.p | 0 |
| e1.y | 8 |
| e2.x | 0 |
| e2.next | 8 |

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```
1   void proc (union ele *up) {
2       up->_________ = *(_________) - _________;
3   }
```

# Example 2

Consider the following union declaration:

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next;
9       } e2;
10  };
```

B. How many total bytes does the structure require?

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```
1   void proc (union ele *up) {
2       up->_________ = *(_________) - _________;
3   }
```

# Example 2

Consider the following union declaration:

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next;
9       } e2;
10  };
```

B. How many total bytes does the structure require?

16

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```
1   void proc (union ele *up) {
2       up->_________ = *(_________) - _________;
3   }
```

# Example 2

Consider the following union declaration:

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next;
9       } e2;
10  };
```

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```
1   void proc (union ele *up) {
2       up->__________ = *(__________) - __________;
3   }
```

C. The compiler generates the following assembly code for `proc`:

```
    void proc (union ele *up)
    up in %rdi
1   proc:
2     movq    8(%rdi), %rax
3     movq    (%rax), %rdx
4     movq    (%rdx), %rdx
5     subq    8(%rax), %rdx
6     movq    %rdx, (%rdi)
7     ret
```

On the basis of this information, fill in the missing expressions in the code for `proc`. *Hint:* Some union references can have ambiguous interpretations. These ambiguities get resolved as you see where the references lead. There is only one answer that does not perform any casting and does not violate any type constraints.

# Example 2

Consider the following union declaration:

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next;
9       } e2;
10  };
```

C. The compiler generates the following assembly code for `proc`:

```
    void proc (union ele *up)
    up in %rdi
1   proc:
2     movq    8(%rdi), %rax
3     movq    (%rax), %rdx
4     movq    (%rdx), %rdx
5     subq    8(%rax), %rdx
6     movq    %rdx, (%rdi)
7     ret
```

On the basis of this information, fill in the missing expressions in the code for `proc`. *Hint:* Some union references can have ambiguous interpretations. These ambiguities get resolved as you see where the references lead. There is only one answer that does not perform any casting and does not violate any type constraints.

This declaration illustrates that structures can be embedded within unions.

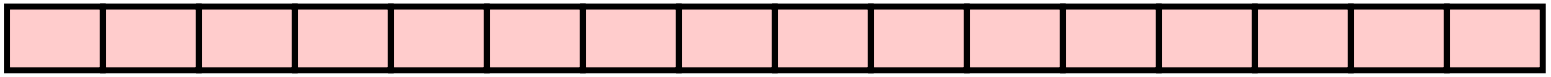
The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```
1   void proc (union ele *up) {
2       up-> e2.x  = *((up→e2.next)→e1.p) - ((up→e2.next)→e1.y) ;
3   }
```

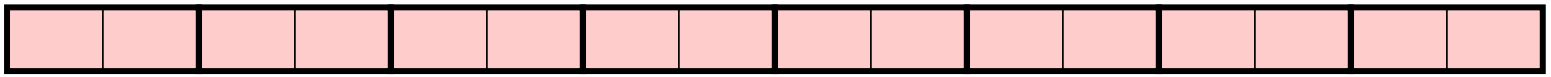
# Floating Point Registers

## XMM Registers

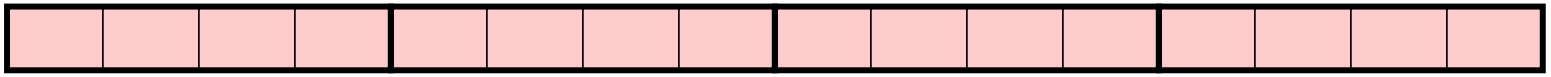
- 16 total, each 16 bytes
- 16 single-byte integers



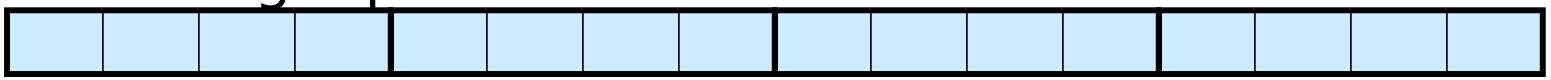
- 8 16-bit integers



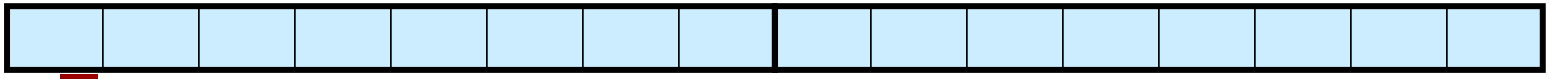
- 4 32-bit integers



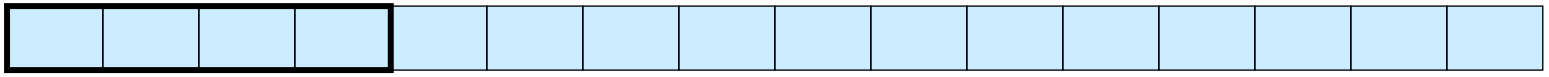
- 4 single-precision floats



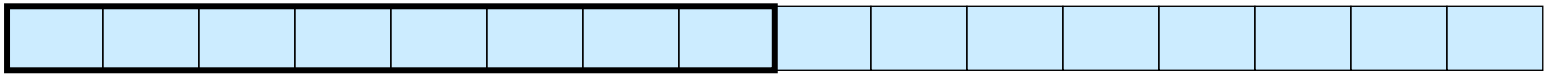
- 2 double-precision floats



- 1 single-precision float

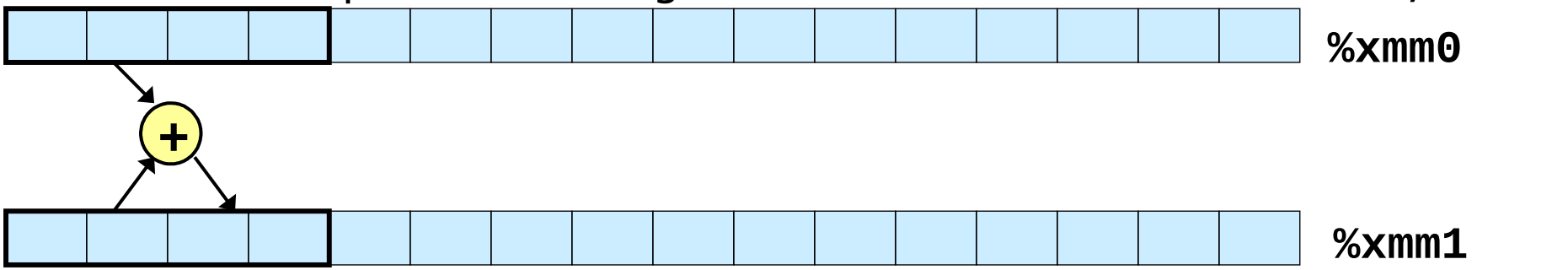


- 1 double-precision float

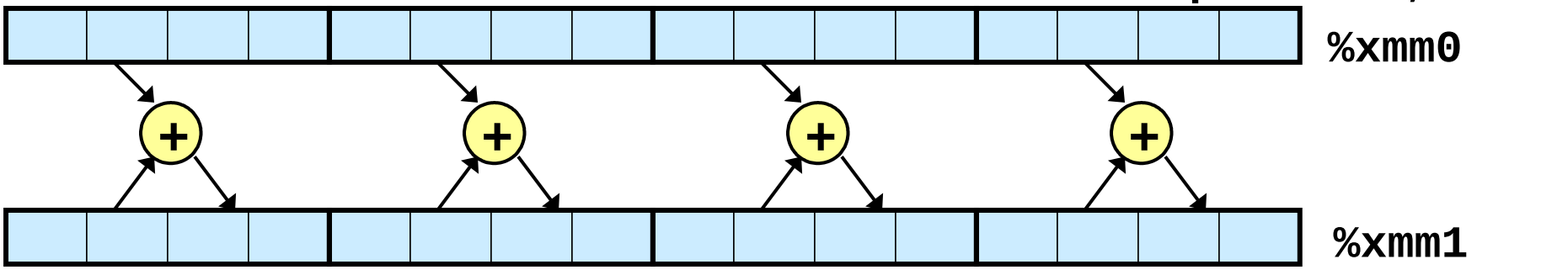


# Scalar & SIMD Operations

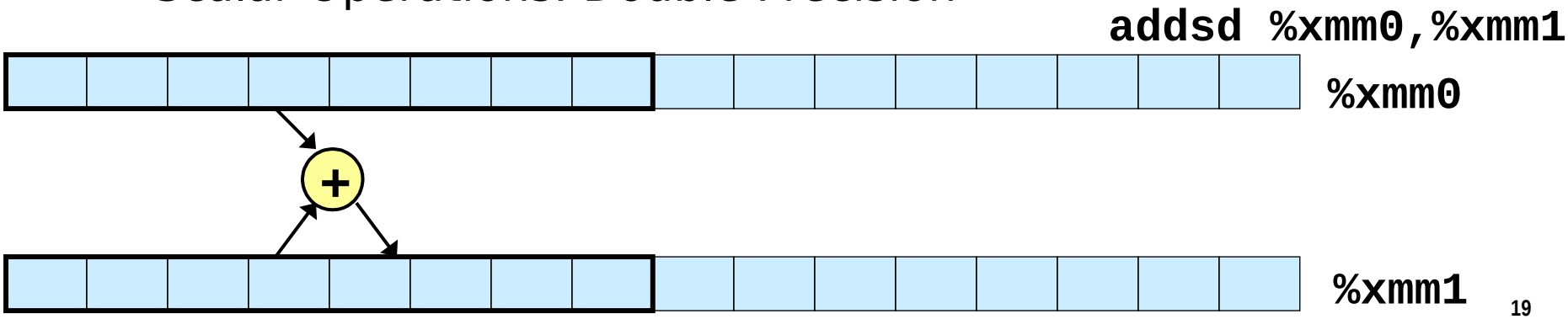
- Scalar Operations: Single Precision




- SIMD Operations: Single Precision




- Scalar Operations: Double Precision



# FP Basics

- Arguments passed in %xmm0, %xmm1, ...
- Result returned in %xmm0
- All XMM registers caller-saved

```
float fadd(float x, float y)
{
    return x + y;
}
```

```
double dadd(double x, double
y)
{
    return x + y;
}
```

```
  # x in %xmm0, y in %xmm1
  addss   %xmm1, %xmm0
  ret
```

```
 # x in %xmm0, y in %xmm1
  addsd   %xmm1, %xmm0
  ret
```

# Example 3

For each of the following function declarations, determine the register assignments for the arguments:

A. `double g1(double a, long b, float c, int d);`

B. `double g2(int a, double *b, float *c, long d);`

C. `double g3(double *a, double b, int c, float d);`

D. `double g4(float a, int *b, float c, double d);`

# Example 3

For each of the following function declarations, determine the register assignments for the arguments:

A. `double g1(double a, long b, float c, int d);`

Registers: a in %xmm0, b in %rdi c in %xmm1, d in %esi

B. `double g2(int a, double *b, float *c, long d);`

C. `double g3(double *a, double b, int c, float d);`

D. `double g4(float a, int *b, float c, double d);`

# Example 3

For each of the following function declarations, determine the register assignments for the arguments:

A. `double g1(double a, long b, float c, int d);`

Registers: a in %xmm0, b in %rdi c in %xmm1, d in %esi

B. `double g2(int a, double *b, float *c, long d);`

Registers: a in %edi, b in %rsi, c in %rdx, d in %rcx

C. `double g3(double *a, double b, int c, float d);`

D. `double g4(float a, int *b, float c, double d);`

# Example 3

For each of the following function declarations, determine the register assignments for the arguments:

A. `double g1(double a, long b, float c, int d);`

Registers: a in %xmm0, b in %rdi c in %xmm1, d in %esi

B. `double g2(int a, double *b, float *c, long d);`

Registers: a in %edi, b in %rsi, c in %rdx, d in %rcx

C. `double g3(double *a, double b, int c, float d);`

Registers: a in %rdi, b in %xmm0, c in %esi, d in %xmm1

D. `double g4(float a, int *b, float c, double d);`

# Example 3

For each of the following function declarations, determine the register assignments for the arguments:

A. `double g1(double a, long b, float c, int d);`

Registers: a in %xmm0, b in %rdi c in %xmm1, d in %esi

B. `double g2(int a, double *b, float *c, long d);`

Registers: a in %edi, b in %rsi, c in %rdx, d in %rcx

C. `double g3(double *a, double b, int c, float d);`

Registers: a in %rdi, b in %xmm0, c in %esi, d in %xmm1

D. `double g4(float a, int *b, float c, double d);`

Registers: a in %xmm0, b in %rdi, c in %xmm1, d in %xmm2